



UNIVERSITY OF
TORONTO

Applied Deep Learning

Lecture 4. *Optimizers and Generalization*

Ali Bereyhi

ECE Department, University of Toronto

Fall 2026



UNIVERSITY OF
TORONTO

Lecture Overview

Quick Recap: *DL Basics*

ML Components

1. *Dataset* which is the collection of samples
2. *Model* $\rightsquigarrow$ We always use *NNs*
3. *Loss* quantifies the difference between model output and *true label*

Training Algorithm

The *training* is performed by minimizing the empirical risk

$$\min_{\mathbf{w}} \hat{R}(\mathbf{w}) = \min_{\mathbf{w}} \frac{1}{I} \sum_{i=1}^I \mathcal{L}(NN_{\mathbf{w}}(\mathbf{x}_i), \mathbf{y}_i) \quad (\text{Training})$$

We use the gradient descent algorithm for this minimization

Recap: *Gradient Descent*

- 1: Initiate at some $\mathbf{w}^{(0)} \in \mathbb{R}^D$ and deviation $\Delta = +\infty$
- 2: Choose some small ε and η , and set $t = 1$
- 3: **while** $\Delta > \varepsilon$ **do**
- 4: Update weights as $\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} - \eta \nabla \hat{R}(\mathbf{w}^{(t-1)})$
- 5: Update the deviation $\Delta = |\hat{R}(\mathbf{w}^{(t)}) - \hat{R}(\mathbf{w}^{(t-1)})|$
- 6: **end while**

Key Computational Challenge

*We need to compute the **gradient of loss** with respect to **all NN weights**!*

Computation Graph

*We can draw NN on a **computation graph** and apply **chain rule***

Recap: *Backpropagation*

ForwardProp():

- 1: Start with input as $\mathbf{y}_0 = \mathbf{x}$
- 2: **for** $\ell = 0, \dots, L$ **do**
- 3: Compute $\mathbf{z}_{\ell+1} = \mathbf{W}_{\ell+1} \mathbf{y}_{\ell}$ # forward affine
- 4: Activate $\mathbf{y}_{\ell+1} = f_{\ell+1}(\mathbf{z}_{\ell+1})$ # forward activation
- 5: **end for**
- 6: Return all $\mathbf{y}_{\ell}$ and $\mathbf{z}_{\ell}$

BackProp():

- 1: Start with loss gradient with respect to output $\hat{\mathbf{y}}_{L+1}$
- 2: **for** $\ell = L, \dots, 1$ **do**
- 3: Compute $\hat{\mathbf{y}}_{\ell} = \mathbf{W}_{\ell+1}^T \hat{\mathbf{z}}_{\ell+1}$ # backward affine
- 4: De-activate as $\hat{\mathbf{z}}_{\ell} = \dot{f}_{\ell}(\mathbf{z}_{\ell}) \odot \hat{\mathbf{y}}_{\ell}$ # backward activation
- 5: **end for**
- 6: **for** $\ell = 1, \dots, L + 1$ **do**
- 7: Return $\nabla_{\mathbf{w}_{\ell}} \hat{R} = \hat{\mathbf{z}}_{\ell} \mathbf{y}_{\ell-1}^T$
- 8: **end for**

Lecture 4. *Optimizers and Generalization*

Today, we study the following topics

1. **Stochastic** gradient descent and its descendants
 - We could train on a realistic dataset using *SGD-like optimizers*
2. SGD descendants
 - How fast does an optimizer converge?
 - We go over *Rprop, RMSprop, and Adam*
3. Overfitting and Regularization
 - What does *generalization* mean?
 - When does a model overfit?
 - Why we need *cross-validation*?
 - What is *regularization*?
 - A few well-known regularization schemes, e.g., *weight decay, Lasso, dropout*



UNIVERSITY OF
TORONTO

Stochastic Gradient Descent

Full-Batch Training via *Gradient Descent*

- + Say we use backpropagation; then, how does *gradient descent* look?
- Well! We should go back and forth with all data-points

Say we have dataset

$$\mathbb{D} = \{(\mathbf{x}_b, \mathbf{v}_b) \text{ for } b = 1, \dots, B\}$$

GradientDescent():

```
1: Initialize with some initial values  $\{\mathbf{W}_\ell^{(0)}\}$  and set a learning rate  $\eta$ 
2: while weights not converged do
3:   for  $b = 1, \dots, B$  do
4:     NN.values  $\leftarrow$  ForwardProp  $(\mathbf{x}_b, \{\mathbf{W}_\ell^{(t)}\})$  # forward
5:      $\{\nabla_{\mathbf{W}_\ell^{(t)}} \hat{R}_b\} \leftarrow$  BackProp  $(\mathbf{x}_b, \mathbf{v}_b, \{\mathbf{W}_\ell^{(t)}\}, \text{NN.values})$  # backward
6:   end for
7:   for  $\ell = 1, \dots, L + 1$  do
8:      $\mathbf{W}_\ell^{(t+1)} \leftarrow \mathbf{W}_\ell^{(t)} - \eta \text{ mean}(\nabla_{\mathbf{W}_\ell^{(t)}} \hat{R}_1, \dots, \nabla_{\mathbf{W}_\ell^{(t)}} \hat{R}_B)$ 
9:   end for
10: end while
```

Full-Batch Training

Batch $\equiv$ *the dataset reserved for training*

In full-batch training, we compute gradients at all data-points in the **batch**: so, we *need to wait till forward and backward pass are over for all B data-points*

Example. *MNIST includes 70,000 images of hand-written numbers with their true labels: from these 70,000 we use 60,000 for training and 10,000 for test*

- ! One iteration of gradient descent requires 60K forward-backward passes
- 🕒 With fair computation, we could process 1K images per second
- ➡ We need 6 seconds per iteration!

This is a huge and unnecessary burdensome!

Sample-Level Training

Simplest solution is to apply one step of **gradient descent** after

each forward-backward pass

SampLevel_GradientDescent():

```
1: Initialize with some initial values  $\{\mathbf{W}_\ell^{(0)}\}$  and set a learning rate  $\eta$ 
2: Start at  $b = 1$ 
3: while weights not converged do
4:   if  $b > B$  then
5:     Update  $b \leftarrow 1$                                 # start over with the dataset
6:   end if
7:   NN.values  $\leftarrow$  ForwardProp ( $\mathbf{x}_b, \{\mathbf{W}_\ell^{(t)}\}$ )
8:    $\{\mathbf{G}_{\ell,b}\} \leftarrow$  BackProp ( $\mathbf{x}_b, v_b, \{\mathbf{W}_\ell^{(t)}\}, \text{NN.values}$ )
9:   Update  $\mathbf{W}_\ell^{(t+1)} \leftarrow \mathbf{W}_\ell^{(t)} - \eta \mathbf{G}_{\ell,b}$           # sample_level update
10:  Update  $b \leftarrow b + 1$                                 # go for next data-point
11: end while
```

Sample-Level Training: *Meaning*

- + *But what does it mean in the sense of empirical risk minimization? Aren't we now doing something different from the **standard gradient descent**?!*
- We are in fact performing an **approximate gradient descent**

Say $\mathbf{G}_b$ contains all derivatives computed after backward pass with $(\mathbf{x}_b, \mathbf{v}_b)$

Consider an **ideal scenario** in which

$$\mathbf{G}_1 = \mathbf{G}_2 = \dots = \mathbf{G}_B$$

In this case, we do not need to wait for the **batch** to be fully over, since

$$\mathbf{G}_1 = \text{mean}(\mathbf{G}_1, \mathbf{G}_2, \dots, \mathbf{G}_B)$$

*In other words, in this case, we **don't really need to wait** for the **full batch***

***sample-level** training $\equiv$ **full-batch** training*

Sample-Level Training: *Formulation*

In practice, at each data-point we compute a *noisy-version* of a *ground truth gradient* $\bar{\mathbf{G}}$: we can think of $\mathbf{G}_b$ for each b as

$$\mathbf{G}_b = \bar{\mathbf{G}} + \text{Noise}$$

If this noise is small enough, we can say that

$$\mathbf{G}_b \approx \bar{\mathbf{G}} \approx \text{mean}(\mathbf{G}_1, \mathbf{G}_2, \dots, \mathbf{G}_B)$$

and therefore, we can conclude that

$$\text{sample-level (worse approx. } \bar{\mathbf{G}}) \approx \text{full-batch (better approx. } \bar{\mathbf{G}})$$

In this case, we say that $\mathbf{G}_b$ is an *estimator* of the *ground truth gradient*

Sample-Level Training: *Repetitive Cycle Issue*

Naive sample-level update can trap us into a **repetitive cycle**: *in simple words, we can end up with our initial point at the end of the batch. For instance, consider the following dummy scenario: in an NN with weights $\mathbf{w}$*

*We start with $\mathbf{w}^{(0)}$ and get into the **batch for the first time***

- We update $\mathbf{w}^{(0)}$ after the **first data-point** to $\mathbf{w}^{(1)}$*
- We update $\mathbf{w}^{(1)}$ after the **second data-point** to $\mathbf{w}^{(2)}$*
- ...*
- We update $\mathbf{w}^{(B-1)}$ after the **last data-point** to $\mathbf{w}^{(B)}$*

Now, assume that $\mathbf{w}^{(B)} = \mathbf{w}^{(0)}$ again!

Further looping over the batch is **useless**, since *we always get back to the initial point: this is the most basic example of the **repetitive cycle issue***

- In practice, this behaviour could occur **approximately***

Stochastic Sample-Level Training: *SGD*

- + How can we avoid such *cyclic behaviors*?
- We can use *Stochastic Gradient Descent (SGD)*

Each time we are to loop over our *training batch*, we *shuffle* the data-points *randomly*: this way we avoid next loop behave like the previous one

This idea is called Stochastic Gradient Descent (SGD)

SGD is the most common algorithm for training of NNs!

What does *random shuffling* mean?

It means *randomly permuting* the data-points

Stochastic Gradient Descent

SGD():

```
1: Initialize with some initial values  $\{\mathbf{W}_\ell^{(0)}\}$  and set a learning rate  $\eta$ 
2: Randomly shuffle the batch and start at  $b = 1$ 
3: while weights not converged do
4:   if  $b > B$  then
5:     Randomly shuffle the batch and set  $b \leftarrow 1$            # random shuffling
6:   end if
7:   NN.values  $\leftarrow$  ForwardProp ( $\mathbf{x}_b, \{\mathbf{W}_\ell^{(t)}\}$ )
8:    $\{\mathbf{G}_{\ell,b}\} \leftarrow$  BackProp ( $\mathbf{x}_b, v_b, \{\mathbf{W}_\ell^{(t)}\}, \text{NN.values}$ )
9:   Update  $\mathbf{W}_\ell^{(t+1)} \leftarrow \mathbf{W}_\ell^{(t)} - \eta \mathbf{G}_{\ell,b}$            # sample_level update
10:  Update  $b \leftarrow b + 1$                                      # go for next data-point
11: end while
```

+ *But, doesn't sample-level training lead to any drawback?*

– Sure! Not too expensive though!

Recap: *Properties of Variance*

For random variable x with mean μ , the **variance** is defined as

$$\text{Var}\{x\} = \mathbb{E}\{(x - \mu)^2\} = \mathbb{E}\{x^2\} - \mu^2$$

1. For any random variable x and constant c , we have

$$\text{Var}\{cx\} = c^2 \text{Var}\{x\}$$

2. Let $x_1, \dots, x_N$ be N **independent** samples; then, we have

$$\text{Var}\left\{\sum_{n=1}^N x_n\right\} = \sum_{n=1}^N \text{Var}\{x_n\}$$

Recap: Variance of Average Estimator

Let $x_1, \dots, x_N$ be N independent samples all with variance σ^2

➡ Define $\bar{x}$ be the arithmetic average of $x_1, \dots, x_N$, i.e.,

$$\bar{x} = \text{mean}(x_1, \dots, x_N) = \frac{1}{N} \sum_{n=1}^N x_n$$

We could then say

$$\begin{aligned} \text{Var}\{\bar{x}\} &= \text{Var}\left\{\frac{1}{N} \sum_{n=1}^N x_n\right\} = \frac{1}{N^2} \text{Var}\left\{\sum_{n=1}^N x_n\right\} \\ &= \frac{1}{N^2} \sum_{n=1}^N \underbrace{\text{Var}\{x_n\}}_{\sigma^2} = \frac{1}{N^2} (N\sigma^2) \\ &= \frac{\sigma^2}{N} \quad \text{variance of average drops by } 1/\# \text{ samples} \end{aligned}$$

Complexity-Accuracy Trade-off of SGD

In practice, at each data-point we calculate a *noisy-version* of a *ground truth gradient* $\bar{\mathbf{G}}$. In other words, we can think of $\mathbf{G}_b$ for each b as

$$\mathbf{G}_b = \bar{\mathbf{G}} + \text{Noise}$$

and called $\mathbf{G}_b$ an *estimator* of the *ground truth*

Let's make the above statement a bit more formal: we assume that Noise for each b is a matrix with *independent zero-mean* entries all with *variance* σ^2 , i.e.,

$$\mathbf{G}_b = \bar{\mathbf{G}} + \mathbf{N}_b$$

where we define $\bar{\mathbf{G}}$ to be the gradient of the *true risk*

$$\bar{\mathbf{G}} = \nabla_{\mathbf{w}} R(\mathbf{w}) = \nabla_{\mathbf{w}} \mathbb{E} \{ \mathcal{L}(y, v) \}$$

Complexity-Accuracy Trade-off of SGD

If you remember, when we started with training

our goal was to minimize the risk $R(\mathbf{w})$

However, we could not do this, since we *did not know*

1. *true function*, and
2. *data distribution*

Thus, we approximated the *true risk $R(\mathbf{w})$* with *empirical risk $\hat{R}(\mathbf{w})$*

We could have used a single sample to approximate the true risk $R(\mathbf{w})$

➡ *and this is intuitively what's done by SGD*

Complexity-Accuracy Trade-off of SGD

In **full-batch** training, we determine the gradient as

$$\begin{aligned}\hat{\mathbf{G}}^{\text{batch}} &= \text{mean}(\mathbf{G}_1, \mathbf{G}_2, \dots, \mathbf{G}_B) = \frac{1}{B} \sum_{b=1}^B (\bar{\mathbf{G}} + \mathbf{N}_b) \\ &= \bar{\mathbf{G}} + \underbrace{\frac{1}{B} \sum_{b=1}^B \mathbf{N}_b}_{\hat{\mathbf{N}}^{\text{batch}}} = \bar{\mathbf{G}} + \hat{\mathbf{N}}^{\text{batch}}\end{aligned}$$

Recall that by *arithmetic averaging variance drops by $1/\#$ of samples*

In **full-batch** training the **approximated gradient** is different from the **true gradient** by an error whose variance **drops as σ^2/B**

Complexity-Accuracy Trade-off of SGD

In *full-batch* training the *approximated gradient* is different from the *true gradient* by an error whose variance *drops as* σ^2/B

Now, let's compare it to SGD

In SGD, we *approximate* the gradient with a sample gradient, i.e.,

$$\hat{\mathbf{G}}^{\text{SGD}} = \mathbf{G}_b = \bar{\mathbf{G}} + \mathbf{N}_b$$

SGD is still *approximating* the *true gradient* but with much *larger variance*

- *Its estimator has* $B \times$ *larger variance* *than full-batch one*

Training via Mini-Batches

- + But, can't we *play with this trade-off*? For instance, *increase* a bit the *complexity* to *improve the accuracy*!
- Yes! This is the idea of *mini-batch training*

In *mini-batch training*, we divide *the whole batch of data* into *mini-batches*

- after each *mini-batch* is over, we *average the gradients* over the *mini-batch*
- we apply *one step of gradient descent* using this *averaged gradient*

To avoid *cyclic behavior*, we still *shuffle* the dataset *randomly* each time we *start a new loop* over it. This training approach is hence often called

Mini-Batch Stochastic Gradient Descent = Mini-Batch SGD

Mini-Batch SGD

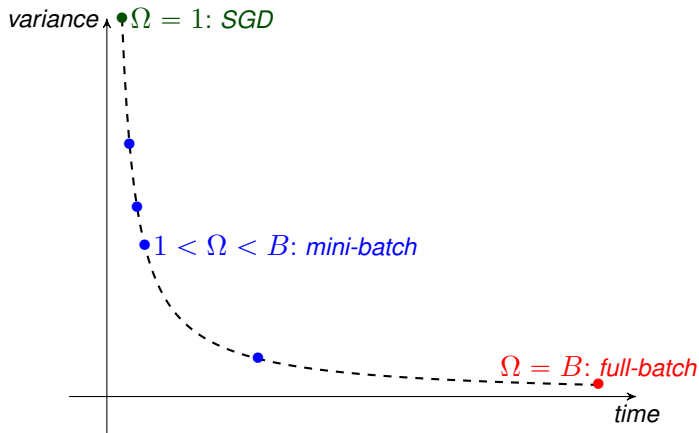
mBatchSGD():

- ```

1: Initialize with some initial values $\{\mathbf{W}_\ell^{(0)}\}$ and set a learning rate η
2: Randomly shuffle the batch and divide it into mini-batches of size Ω
3: Denote the number of mini-batches by $\Xi = \lceil B/\Omega \rceil$ and start at $\xi = 1$
4: while weights not converged do
5: if $\xi > \Xi$ then
6: Randomly shuffle the batch and divide it into mini-batches of size Ω
7: Set $\xi \leftarrow 1$ # start over with the dataset
8: end if
9: for $\omega = 1, \dots, \Omega$ do
10: NN.values $\leftarrow$ ForwardProp ($\mathbf{x}_\omega, \{\mathbf{W}_\ell^{(t)}\}$)
11: $\{\mathbf{G}_{\ell,\omega}\} \leftarrow$ BackProp ($\mathbf{x}_\omega, v_\omega, \{\mathbf{W}_\ell^{(t)}\}, \text{NN.values}$)
12: end for
13: Update $\mathbf{W}_\ell^{(t+1)} \leftarrow \mathbf{W}_\ell^{(t)} - \eta \text{ mean}(\mathbf{G}_{\ell,1}, \dots, \mathbf{G}_{\ell,\Omega})$
14: Update $\xi \leftarrow \xi + 1$ # go for next mini-batch
15: end while

```

# Complexity-Accuracy Trade-off



*Mini-batch size is what specifies the trade-off point*

## Few Definitions: *Epoch and Iteration*

### Batch Size

Through time, the term *mini-batch* has been transformed to *batch*, and *complete batch* is referred to as *train-set* or the *full batch*. People hence call the size of each *mini-batch*, i.e.,  $\Omega$ , the *batch size*

### Iteration

When we take one step of gradient descent, we take one *iteration*. So, one *iteration* is over when we finish with a *single mini-batch*

### Epoch

An *epoch* is over when we finish *once* with the *whole train-set*

## Few Definitions: *Epoch and Iteration*

*We can annotate these definitions in our algorithm*

mBatchSGD():

- 1: Initialize with some initial values  $\{\mathbf{W}_\ell^{(0)}\}$  and set a learning rate  $\eta$
- 2: Randomly shuffle **training dataset** and make mini-batches of size  $\Omega \equiv \text{batch-size}$
- 3: Denote the number of mini-batches by  $\Xi = \lceil B/\Omega \rceil$  and start at  $\xi = 1$
- 4: **while** weights not converged **do**
- 5:   **if**  $\xi > \Xi$  **then**
- 6:     Randomly shuffle the batch and divide it into mini-batches of size  $\Omega$
- 7:     Set  $\xi \leftarrow 1$      ← one epoch is over, we start another epoch
- 8:   **end if**
- 9:   **for**  $\omega = 1, \dots, \Omega$  **do**
- 10:     NN.values  $\leftarrow$  ForwardProp ( $\mathbf{x}_\omega, \{\mathbf{W}_\ell^{(t)}\}$ )     going through a min-Batch
- 11:      $\{\mathbf{G}_{\ell,\omega}\} \leftarrow$  BackProp ( $\mathbf{x}_\omega, v_\omega, \{\mathbf{W}_\ell^{(t)}\}, \text{NN.values}$ )
- 12:   **end for**
- 13:   Update  $\mathbf{W}_\ell^{(t+1)} \leftarrow \mathbf{W}_\ell^{(t)} - \eta \text{ mean } (\mathbf{G}_{\ell,1}, \dots, \mathbf{G}_{\ell,\Omega}) \leftarrow$  one iteration
- 14:   Update  $\xi \leftarrow \xi + 1$      # go for next mini-batch
- 15: **end while**

## Few Definitions: *Epoch and Iteration*

Say we train over MNIST using *mini-batch SGD* with *batch size*  $\Omega = 100$ . Our *train-set* has 60,000 samples; thus, we have

$$\Xi = \frac{60,000}{100} = 600$$

*mini-batches*. Each time we finish with a *mini-batch*, we do *one iteration* of gradient descent. After *600 iterations*, we finish with a single *epoch*

So, if we train for 10 *epochs*

we do  $600 \times 10 = 6000$  *iterations of gradient descent*

## Testing NNs with New Data-Point

- + Say we are over with the *training*; then, what should we do?
- We need to test it with the data we reserved for *testing*

We can test NN for a new *test data-point* by evaluating *classical metrics*

1. **Loss.** We pass  $x_{\text{new}}$  forward through our *trained NN* and get  $y_{\text{new}}$ . We then compute the *test loss* as  $\mathcal{L}(y_{\text{new}}, v_{\text{new}})$
2. **Accuracy.** We use  $y_{\text{new}}$  to classify  $x_{\text{new}}$  and compare it to *true class*  $v_{\text{new}}$

Testing for a single new point is *not reliable*: we reserve a *test-set* and test over it

- In MNIST, we only use 60K out of all 70K samples for training
  - The remaining 10K are used to test our model
- ! We *never* expose our model to *test data during training*

## Learning Curves

In practice, the SGD can take **very long** to converge, i.e., to stop iterating

*it needs **too many iterations** to get **too close** to the minimum*

But, it might be **not really needed** to get **that close**! So,

*we test our NN once every **couple of epochs***

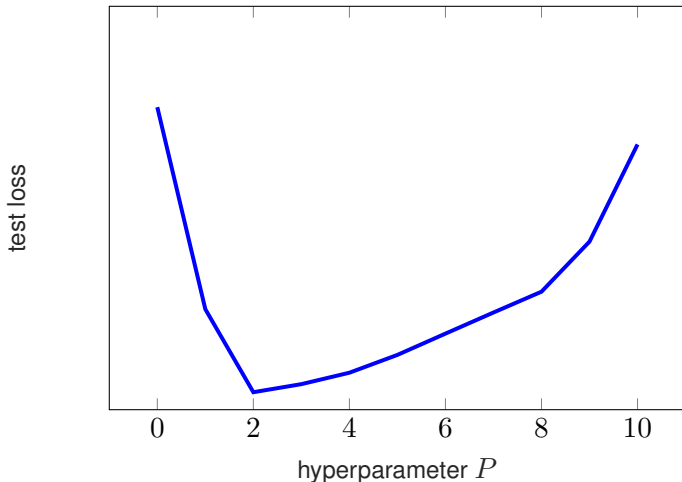
We then plot the **test loss** and **test accuracy** against **number of epochs** in a curve:  
*these curves are often called **learning curves***

*if we see that learning curves are **saturating**, we can **stop** a training*

In practice: we always **train** for a **fixed number of epochs**

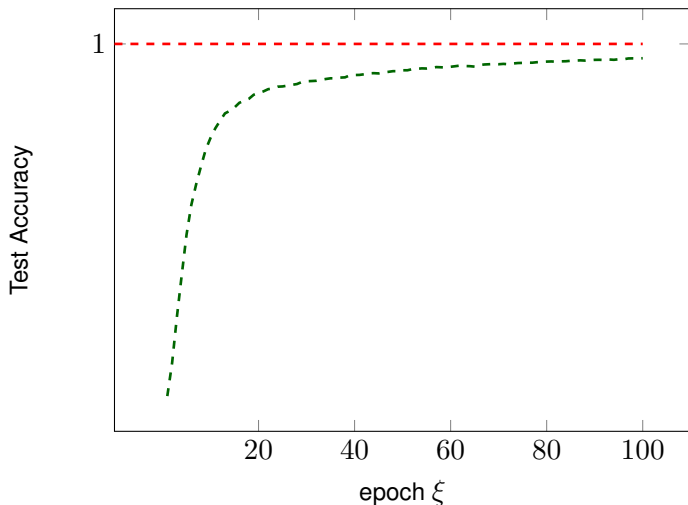
## Learning Curves: *Test Loss*

*How should the learning curves look? A typical curve for test loss is*



## Learning Curves: *Test Accuracy*

*How should the learning curves look? A typical curve for test accuracy is*





UNIVERSITY OF  
TORONTO

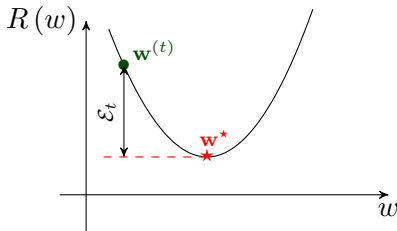
## More on Optimizers

# Optimality Gap

## Optimality Gap in Iteration $t$ (informal)

Optimality gap is the gap between  $\mathbf{w}^{(t)}$  and local minimizer  $\mathbf{w}^*$ , i.e.,

$$\mathcal{E}_t = \text{optimality gap in iteration } t = |R(\mathbf{w}^{(t)}) - R(\mathbf{w}^*)|$$



## Convergence of Optimizers

An optimizer *shrinks this gap gradually*, i.e., we can *approximately* say

$$|R(\mathbf{w}^{(t+1)}) - R(\mathbf{w}^*)| \leq |R(\mathbf{w}^{(t)}) - R(\mathbf{w}^*)| \leq \dots \leq |R(\mathbf{w}^{(0)}) - R(\mathbf{w}^*)|$$

*Ideal case* is that the gap *drops by an  $\alpha < 1$  in each iteration*

$$\begin{aligned}\mathcal{E}_t = |R(\mathbf{w}^{(t)}) - R(\mathbf{w}^*)| &\leq \alpha |R(\mathbf{w}^{(t-1)}) - R(\mathbf{w}^*)| \\ &\leq \alpha^2 |R(\mathbf{w}^{(t-2)}) - R(\mathbf{w}^*)| \\ &\leq \dots \\ &\leq \alpha^t |R(\mathbf{w}^{(0)}) - R(\mathbf{w}^*)|\end{aligned}$$

Now say that  $|R(\mathbf{w}^{(0)}) - R(\mathbf{w}^*)| = C$ ,<sup>1</sup> we can say

$$\mathcal{E}_t \leq C\alpha^t \rightsquigarrow \mathcal{O}(\alpha^t)$$

---

<sup>1</sup>We don't really care how large  $C$  is. It's going to shrink at the end

## Convergence of Optimizers: *Linear Convergence*

If we wish to *end up somewhere* in  $\epsilon$ -neighborhood of  $\mathbf{w}^*$ ; then, we need

$$\mathcal{E}_t \leq C\alpha^t \leq \epsilon$$

For this to happen, we *need to have at least*

$$t \geq \frac{\log 1/\epsilon + \log C}{\log 1/\alpha} \rightsquigarrow \mathcal{O}(\log 1/\epsilon) \equiv \mathcal{O}(-\log \epsilon)$$

*iterations: the closer we need to get, the more we should iterate*

*with this time order, we say that the optimizer converges linearly*

*It's a fast rate, since number of iterations is proportional to logarithm of  $1/\epsilon$*

## Convergence of Optimizers: *SGD*

+ You said *ideal!* Isn't gradient descent *always* converging at *this rate*?

– Well! Only when we optimize *full-batch risk* which is *strongly convex!*

With *realistic SGD*, we move in each iteration *stochastically*

$$\mathcal{E}_t = \text{gap of full-batch risk to its minimum} + \text{stochastic error}$$

We hence define the optimality gap statistically

### Expected Optimality Gap (*informal*)

With a *SGD-like* optimizer, we define the *expected* optimality gap as

$$\bar{\mathcal{E}}_t = \mathbb{E} \{ \mathcal{E}_t \} = \underbrace{\mathcal{E}_t^*}_{\text{full-batch gap}} + \underbrace{\Delta_t}_{\text{estimation error}}$$

## Convergence of SGD

When loss is **smooth functions**<sup>2</sup>, we have for SGD

$$\bar{\mathcal{E}}_t \leq \frac{L}{t\eta} + C\eta\sigma^2$$

for some  $L$  and  $C$  that depend on loss and model, and  $\sigma^2$  that depends on data

- $L$  and  $C$  somehow increase by **model size and sensitivity of loss**
- $\sigma^2$  somehow drops with the **gradient variance**

## Key Observation

SGD-like optimizers need to be tuned in practice

- large **learning rates** can improve **speed** but increase **estimation error**
- shrinking **gradient variance** reduces **estimation error** and hence helps **speed**

---

<sup>2</sup>For example, it's **Lipschitz** continuous

## Convergence of SGD: *Sublinear Convergence*

Note that even without **estimation error**

SGD *converges sub-linearly*

Say, we ignore **estimation error**: to end up in  $\varepsilon$ -neighborhood of  $\mathbf{w}^*$ , we need

$$t = \mathcal{O}(1/\varepsilon)$$

This can potentially **take much much longer** than *linear case*!

+ How **bad** can it be?

– Just set  $\varepsilon = 10^{-3}$

↳ with *linear* convergence we need **time** in order of **3**

↳ with *sub-linear* one, we need in order of **1000**!

## Alternative Optimizers

In practice, we often use **improved** versions of **SGD**

- SGD with *learning rate scheduling*
- SGD with *momentum*
- *Rprop: Resilient backpropagation*
- *RMSprop: Root mean square propagation*
- *Adam: Adaptive moment estimation*

## SGD-like Optimizers

These approaches are all SGD-like: *they*

- *tune learning rate for a fair trade-off between gap and error*
- *control the gradient variance to shrink estimation error*

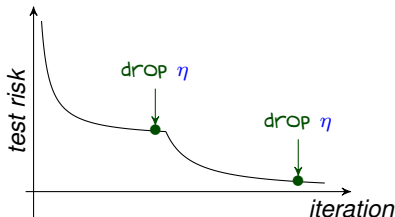
## SGD with Learning Rate Scheduling

What if we shrink **learning rate** as  $\eta \propto 1/\sqrt{t}$

$$\bar{\mathcal{E}}_t \leq \frac{L}{t\eta} + C\eta\sigma^2 \rightsquigarrow \mathcal{O}(1/\sqrt{t})$$

This suggest that we may schedule **learning rate** with **time**: *classical approach*

1. Use a **fixed**  $\eta$  until the **test risk** saturates
2. Reduce  $\eta$  as  $\eta \leftarrow \alpha\eta$  for  $\alpha < 1$ , e.g.,  $\alpha = 0.1$



## Momentum: *Gradient Moving Average*

We can reduce **gradient variance** by computing a **moving average**!

### Momentum

At time  $t$  **momentum** with factor  $\beta$  is computed from **sample gradient**  $\mathbf{g}^{(t)}$

$$\mathbf{m}^{(t)} = \beta \mathbf{m}^{(t-1)} + (1 - \beta) \mathbf{g}^{(t)}$$

We can iterate with **momentum** instead of **sample gradient**

```
Initialize $\mathbf{m}^{(0)} = \mathbf{0}$
for $t = 1, \dots$ do
 :
 $\mathbf{m}^{(t)} = \beta \mathbf{m}^{(t-1)} + (1 - \beta) \mathbf{g}^{(t)}$
 $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta^{(t)} \mathbf{m}^{(t)}$
 :
end for
```

## Nesterov Momentum: *Accelerated Gradient Computation*

Nesterov approach adds an *intermediate* step: when we compute *gradient*, we use the *already-calculated momentum* to *improve* the *estimate of gradient*

```
Initiate $\mathbf{m}^{(0)}$
for $t = 1, \dots$ do
 $\vdots$
 $\hat{\mathbf{w}} = \mathbf{w}^{(t)} - \beta \mathbf{m}^{(t-1)}$ # approximate next point
 $\hat{\mathbf{g}}^{(t)} \leftarrow$ sample gradient at $\hat{\mathbf{w}}$
 $\mathbf{m}^{(t)} = \beta \mathbf{m}^{(t-1)} + (1 - \beta) \hat{\mathbf{g}}^{(t)}$
 $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta^{(t)} \mathbf{m}^{(t)}$
 $\vdots$
end for
```

Typical choice of *momentum factor* is  $\beta = 0.9$ , and remember that

with  $\beta = 0$ , we *return* to the standard *SGD*

## Rprop: Resilient Backpropagation

Rprop was introduced by *Riedmiller and Braun* in 1992; [check the paper here](#)

↳ An approach to schedule *learning rate* for *each weight individually*

```
Rprop():
 Initialize η and choose $\mu^+ > 1, \mu^- < 1, \eta_{\max}$ and $\eta_{\min}$
 for $t = 1 : T$ do
 :
 $\mathbf{g}^{(t)} \leftarrow$ sample gradient at $\mathbf{w}^{(t)}$
 Update learning rate as $\eta \leftarrow \text{Rprop_Scheduler}(\eta, \mathbf{g}^{(t)}, \mathbf{g}^{(t-1)})$
 Update weight $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \odot \text{sign}(\mathbf{g}^{(t)})$ # only sign of gradient
 :
 end for
```

Note that  $\eta$  is a vector with the same size as  $\mathbf{w}$  and  $\mathbf{g}$

## Rprop: Learning Rate Scheduler

The key point of *Rprop* is its *scheduler*

```
Rprop_Scheduler():
```

```
 Use $\mu^+ > 1$ and $\eta_{\max}$ as well as $\mu^- < 1$ and $\eta_{\min}$
```

```
 for each dimension i do
```

```
 if $\text{sign}(g_i^{(t)}) = \text{sign}(g_i^{(t-1)})$ then
```

```
 Update learning rate $\eta_i = \min \{ \mu^+ \eta_i, \eta_{\max} \}$
```

# go faster

```
 else if $\text{sign}(g_i^{(t)}) \neq \text{sign}(g_i^{(t-1)})$ then
```

```
 Update learning rate $\eta_i = \max \{ \mu^- \eta_i, \eta_{\min} \}$
```

# slow down

```
 end if
```

```
 end for
```

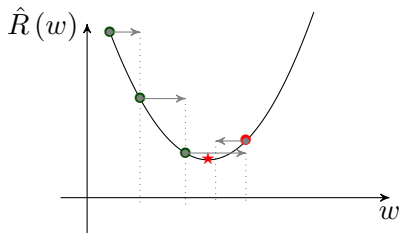
It follows an *intuitive* strategy

↳ Keep *going faster* as the *sign of gradient is not changing*

↳ *Slow down* only when *the sign changes*: you have *passed the minimum!*

## Rprop: *Resilient Backpropagation*

Let's look at it geometrically



In simple words, with **Rprop**

we keep on *increasing* until we *pass the minimum*

## RMSprop: Root Mean Square Propagation

It turns out the **Rprop** works fine with full-batch training

↳ it's because, it **ignores** the **magnitude of gradient**

Consider simple case of  $\mathbf{w} = w$ : we break the **train-set** into 4 **mini-batches** and get the following **sample derivatives** in **each mini-batch**

$$(1) \longleftrightarrow 0.1 \quad (2) \longleftrightarrow 0.1 \quad (3) \longleftrightarrow 0.1 \quad (4) \longleftrightarrow -0.5$$

We may **approximately** say that **full-batch derivative** is **close to zero** or **negative**; however, **Rprop**

1. takes **first three steps** with **larger and larger learning rates**
2. only **comes back** with **smaller step** at **last iteration**

It could be hence **already lost** at the **last iteration**!

## RMSprop: Root Mean Square Propagation

**Geoffrey Hinton** came up with a *solution*: we can use the idea of *moving average* to *further normalize* the *learning rate* according to *average gradient magnitude*

this way we do *not completely ignore* the *magnitude of gradient*

---

Hinton looks differently at *update rule* of *Rprop*: recall the *update rule*

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \odot \text{sign}(\mathbf{g}^{(t)})$$

We can write *alternatively* as

learning rates are updated entry-wise

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \odot \frac{\mathbf{g}^{(t)}}{|\mathbf{g}^{(t)}|}$$

where  $|\cdot|$  operates *entry-wise*

## RMSprop: *Root Mean Square Propagation*

Intuitively, RMSprop is SGD with Rprop-inspired learning rate scheduling

Starting with some  $\mathbf{v}^{(0)} = \mathbf{0}$  we determine in iteration  $t$

$$\mathbf{v}^{(t)} = \beta \mathbf{v}^{(t-1)} + (1 - \beta) |\mathbf{g}^{(t)}|^2$$

for some  $\beta < 1$  and normalize  $\mathbf{g}^{(t)}$  with  $\sqrt{\mathbf{v}^{(t)}}$   $\rightsquigarrow$  we can then use scalar  $\eta$

Initialize  $\mathbf{v}^{(0)} = \mathbf{0}$

for  $t = 1 : T$  do

⋮

$$\mathbf{v}^{(t)} = \beta \mathbf{v}^{(t-1)} + (1 - \beta) |\mathbf{g}^{(t)}|^2$$

# compute moving average

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \frac{\mathbf{g}^{(t)}}{\sqrt{\mathbf{v}^{(t)}}}$$

# normalize by RMS

⋮

end for

## Adam Optimizer

We could further drop **gradient variance** in **RMSprop** using **momentum**: *this is*

*Adaptive Momentum Estimation: Adam*

that was proposed by *Kingma and Ba in 2015*<sup>3</sup>

Initiate  $\mathbf{m}^{(0)}$  and  $\mathbf{v}^{(0)}$

**for**  $t = 1 : T$  **do**

$\vdots$

$$\mathbf{m}^{(t)} = \beta_1 \mathbf{m}^{(t-1)} + (1 - \beta_1) \mathbf{g}^{(t)}$$

# compute **momentum**

$$\mathbf{v}^{(t)} = \beta_2 \mathbf{v}^{(t-1)} + (1 - \beta_2) |\mathbf{g}^{(t)}|^2$$

# compute **RMS**

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \frac{\mathbf{m}^{(t)}}{\sqrt{\mathbf{v}^{(t)}}}$$

# **move** with **normalized momentum**

$\vdots$

**end for**

<sup>3</sup>Click to check out the original paper!



UNIVERSITY OF  
TORONTO

# Overfitting and Regularization

## Example: *Fitting Polynomial from Noisy Data*

We have a **machine** which gets **real-valued**  $x$  and **returns**

$$y = x^2 + 3x + 3$$

To learn the input-output relation, we measure the **output** for a **batch of inputs**. Our **measurements** are however **noisy**, i.e.,

$$v_b = x_b^2 + 3x_b + 3 + \varepsilon_b$$

where  $\varepsilon_b$  is **noise** with **bounded** magnitude, i.e.,  $|\varepsilon_b| \leq \alpha$  for some constant  $\alpha$

### Data

The dataset contains **inputs** and their corresponding **noisy outputs**

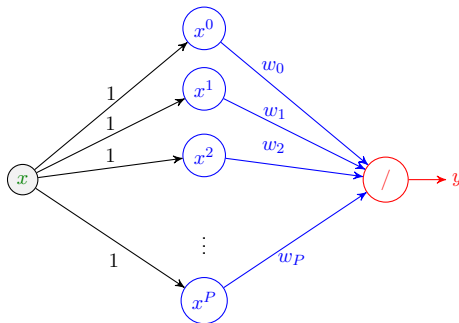
$$\mathbb{D} = \{(x_b, v_b) : i = 1, \dots, B\}$$

## Example: *Polynomial Fitting - Model*

We know that **machine** is **polynomial**: we assume a **polynomial model**

$$y = w_0 + w_1x + w_2x^2 + \dots + w_Px^P$$

for some **integer order**  $P \rightsquigarrow$  a **shallow NN**



Key **hyperparameter** in this **network** is  $P \rightsquigarrow$  decides the **strength** of the model

## Example: *Polynomial Fitting - Loss*

We use *squared error* as the *loss* function, i.e.,

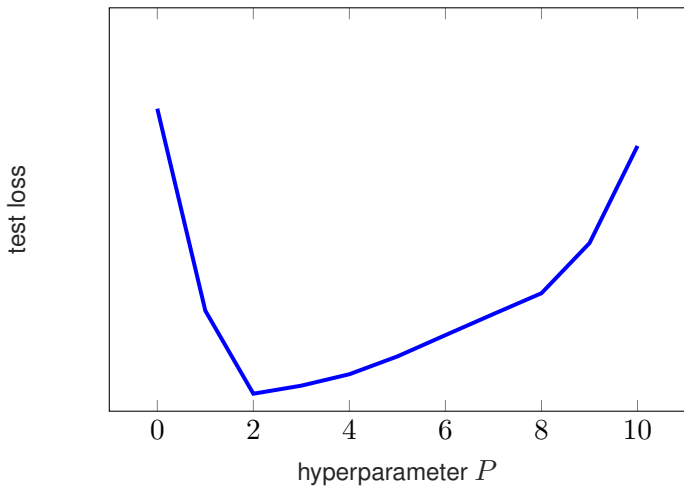
$$\mathcal{L}(y, v) = (y - v)^2$$

*Naive conclusion* might be that *making the NN large* is *always good*; if so

when we *increase  $P$* , we should always see *lower test loss*

However, it's *not* the case!

## Example: *Polynomial Fitting*



# Over- and Underfitting

## Overfitting

*Overfitting* occurs when *training* fits the model, i.e., *NN*, to the *training dataset*, so that it does *not generalize* to *new data-points*

We may also *pay attention* to the term *generalize* in this definition

## Generalization

We say a *model*, i.e., *NN*, *generalizes* well if it achieves similar performance on *unseen data* as on its *training data*

The model might also underfit

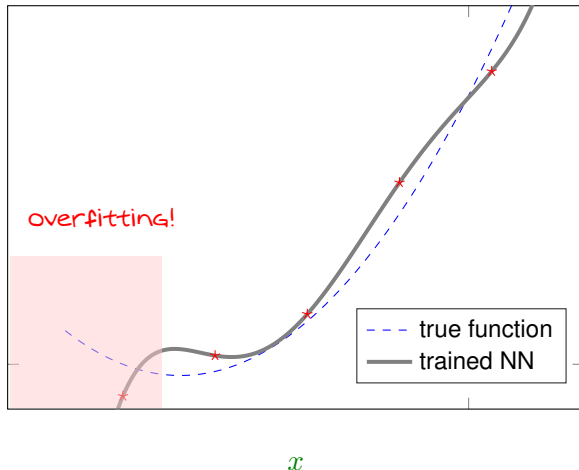
## Underfitting

*Underfitting* occurs when the *model*, i.e., *NN*, *neither* fits to the *training* dataset, *nor generalizes* to *new data*

# Overfitting: Polynomial Fitting

With large  $P$  the NN fits perfectly to training data, but does not generalize

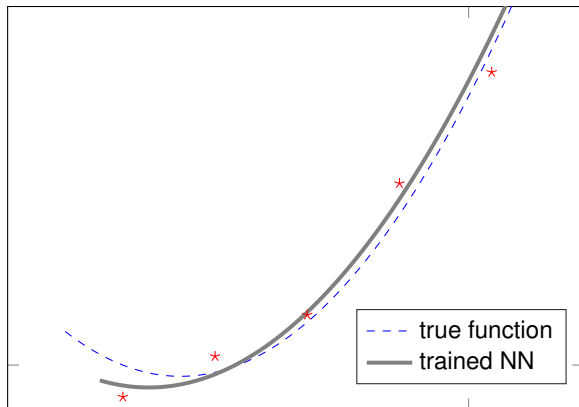
label  $y$  and NN output  $y$



# Overfitting: *Polynomial Fitting*

With  $P = 2$  the NN fits good to training data, and generalizes

label  $y$  and NN output  $\hat{y}$

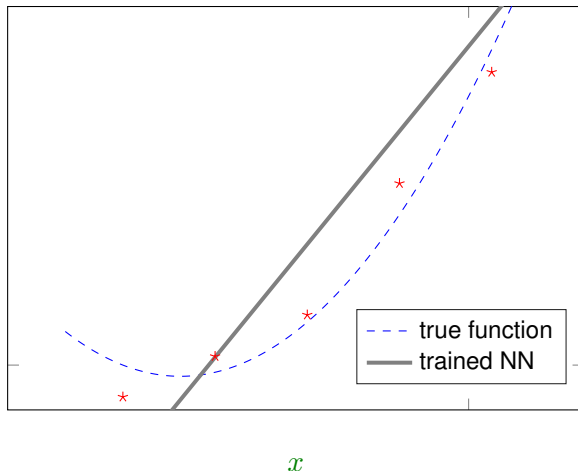


$x$

## Underfitting: *Polynomial Fitting*

With  $P < 2$  the NN neither fits *training data*, nor *generalizes*

label  $y$  and NN output  $\hat{y}$



## Why Overfitting Happens

There are various **approaches** to handle **overfitting**

- *Regularization*
- *Dropout*
- *Data Augmentation*
- ...

To understand *these approaches*, we should first answer *the following question*

*When does **overfitting** happen?*

## Why Overfitting Happens: *Model Capacity*

We know the *initial answer*: *in our dummy example*, it happened because  
we assumed *large polynomial order*

In other words *our model* was *too complex* for our learning task

We can *extend this to NNs*: *overfitting* happens when *the model* is *too complex*,  
*i.e.*, it's suited for learning *complicated functions*

*When does overfitting happen?* It happens when

1. *the model* has a *large capacity*

### Model Capacity

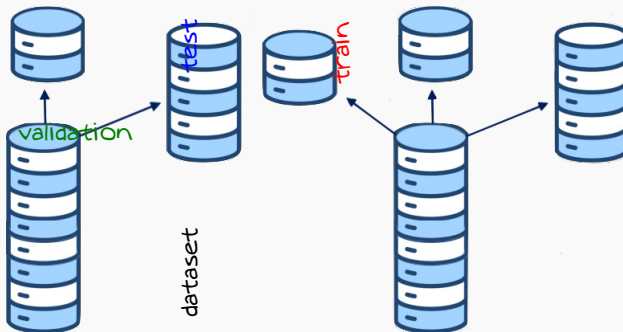
*Size or complexity* of the *set of functions* that a model can represent  $\propto$  *model size*

## Cross-Validation

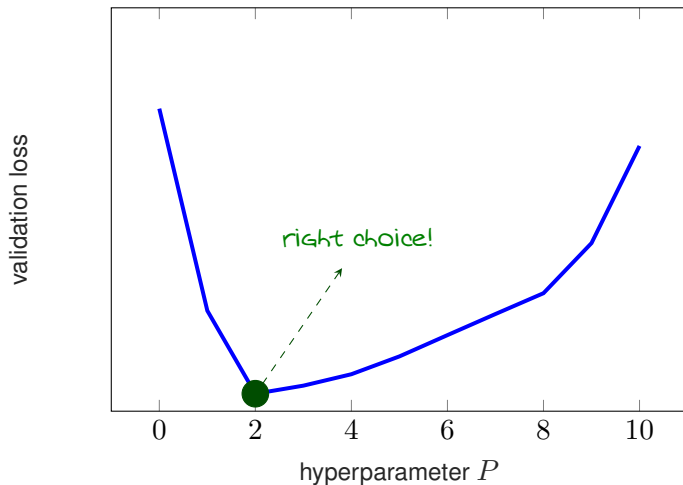
We use **cross-validation** to tune the **hyperparameters**  $\rightsquigarrow$  control **model capacity**

## Holdout Validation

We evaluate model on a **validation set** separated from **train** and **test sets**



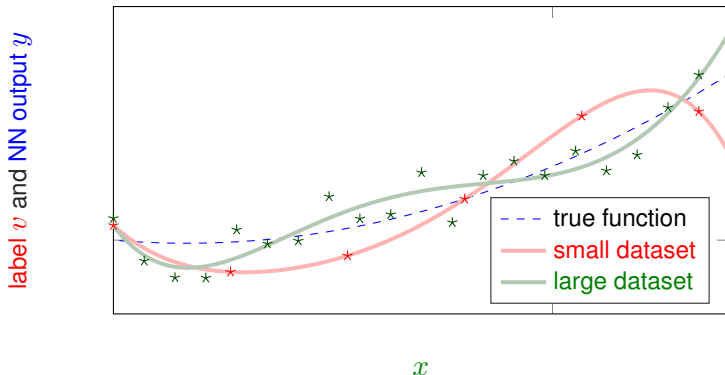
## Validation: *Polynomial Fitting*



## Why Overfitting Happens: *Dataset Size*

In *our example*, consider an *overfitted model* with  $P = 5$  and *train* it on

1. a *small dataset* with only 4 data-points
2. a *large dataset* with 20 data-points



## Why Overfitting Happens: *Dataset Size*

This is a **general behavior** . . .

*if we have a **large enough dataset** model **cannot really overfit** too much*

*When does **overfitting** happen? It happens when*

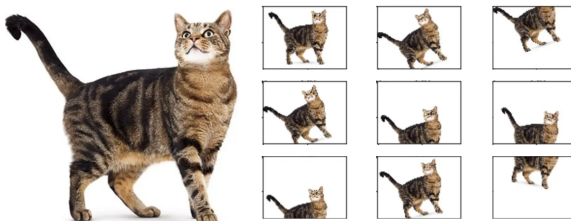
- 1. **the model** has a **large capacity***
- 2. our training **dataset** is **small***

## Mutually Related Sources

*For a **large-capacity model**, e.g., **too deep or wide**, training data can be **insufficient** leading to **extreme overfitting***

## Data Augmentation

With visual data, we can add new samples by *rotate*, *shift*, *zoom-in* and so on!



We can think of a generic pipeline for augmenting from available samples

$$\text{training dataset } \mathbb{D} \rightsquigarrow \mathcal{A}(\cdot) \rightsquigarrow (\mathbf{x}_{\text{new},1}, \mathbf{u}_{\text{new},1}), (\mathbf{x}_{\text{new},2}, \mathbf{u}_{\text{new},2}), \dots$$

## Data Augmentation

A data augmentation technique  $\mathcal{A}(\cdot)$  takes the *training dataset* as the input and returns *new samples* from *data distribution*

## Data Augmentation by *Engineering*

Data augmentation by engineering depends on data and learning task

- If we are classifying a set of images
  - ↳ We can apply geometric transformations, e.g., random rotating, flipping, etc
  - ↳ We may use kernel filters to make random filtering, e.g., blurring
  - ↳ We can randomly mix images, e.g., mix multiple cat images
- If we are working with audio signals
  - ↳ We can apply noise injection, e.g., add background noise
  - ↳ We may change sampling rate to change the speed of audio data
  - ↳ We can apply random shifts, e.g., sample signals at a bit deviated points
- If we are dealing with text data
  - ↳ We may apply random replacements, e.g., replace a word with its synonyms
  - ↳ We may manipulate syntax-tree, e.g., rephrase a sentence
  - ↳ We may apply removal and insertion of redundant words, e.g., so

## Synthetic Data Generation

Recall Assignment 1  $\rightsquigarrow$  we knew by Newton's laws that distance  $d$  is given by

$$d = 0.45v\sqrt{h}$$

To make dataset, we generated lots of velocities  $v_i$  and heights  $h_i$  at random and for each pair we determined  $d_i$  by above equation

---

We can think of a generic pipeline for augmenting from random seeds

$$\text{random seed } s \rightsquigarrow \boxed{\mathcal{S}(\cdot)} \rightsquigarrow (\mathbf{x}_1, v_1), (\mathbf{x}_2, v_2), \dots$$

## Synthetic Data Generation

A synthetic data generator  $\mathcal{S}(\cdot)$  takes a random seed as the input and returns samples from data distribution

## Why Overfitting Happens: *Co-Adaptation*

In the example, say we set  $P = 5$  and train on 8 samples: we take a look at *the trained weights* at different iterations  $\rightsquigarrow$  recall that *model parameters* are

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_5 \end{bmatrix}$$

We start with *vector of all zeros* and keep on going

## Why Overfitting Happens: *Co-Adaptation*

Note that the *ground truth*  $\mathbf{w}^*$  for our polynomial machine

$$\mathbf{w}^* = \begin{bmatrix} 3 \\ 3 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Now, let's look at few iterations

$$\mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \rightsquigarrow \mathbf{w}^{(t)} = \begin{bmatrix} 2.61 \\ 2.36 \\ 0.71 \\ 0.01 \\ 0.02 \\ 0.01 \end{bmatrix} \rightsquigarrow \mathbf{w}^{(t')} = \begin{bmatrix} 3.03 \\ 2.97 \\ 0.98 \\ 0.21 \\ 0.12 \\ 0.09 \end{bmatrix}$$

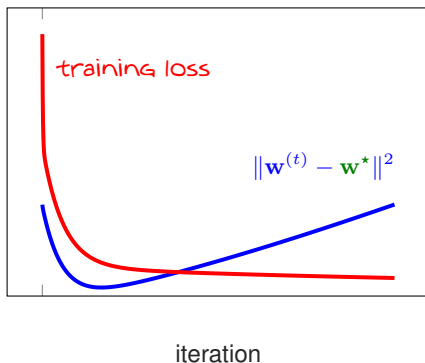
$\mathbf{w}^{(t')}$  looks good! But, what if we keep on training

$$\mathbf{w}^{(t')} = \begin{bmatrix} 3.03 \\ 2.97 \\ 0.98 \\ 0.21 \\ 0.12 \\ 0.09 \end{bmatrix} \rightsquigarrow \dots \rightsquigarrow \mathbf{w}^{(t'')} = \begin{bmatrix} 2.36 \\ 4.43 \\ 3.13 \\ -2.1 \\ 1.98 \\ -1.2 \end{bmatrix}$$

## Why Overfitting Happens: *Co-Adaptation*

Weights start to get *close to what we want* up to *intermediate number of iterations*  $t'$ . But, by *further training* they start to *deviate*  $\equiv$  *overfit*

We can also see this behavior in the figure below



## Why Overfitting Happens: Co-Adaptation

*This behavior is **co-adaptation** of the **parameters***

In initial iterations, NN fits to the true model: **first iterations** of SGD

*update **majorly**  $w_0$ ,  $w_1$  and  $w_2$*

After NN has **gone** close to **ground truth**, it starts to **overfit**:

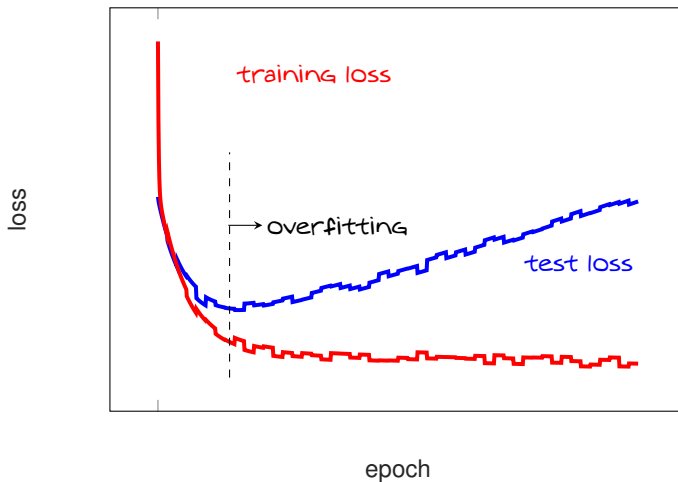
*$w_3$ ,  $w_4$  and  $w_5$  try to **co-adapt**, i.e., **compensate the gap** caused by **noise***

### Intuition on Co-Adaptation

*In **first iterations** NN learns **true function**, later it learns **noise**!*

***Co-adaptation** is the **most observable implication** of **overfitting** with NNs*

## Co-Adaptation: *Typical Learning Curve*



## Why Overfitting Happens: *Final List*

*When does overfitting happen? It happens when*

1. *the model has a large capacity*
2. *our training dataset is small*
3. *parameters start to co-adapt*

## Mutually Related Sources

*For a large-capacity model, e.g., too deep or wide, training data can be insufficient leading to extreme overfitting, especially if we keep training for too many epochs!*

## Training by Penalized Risk: *Regularization*

In our particular example with

$$\mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \rightsquigarrow \mathbf{w}^{(t')} = \begin{bmatrix} 3.03 \\ 2.97 \\ 0.98 \\ 0.21 \\ 0.12 \\ 0.09 \end{bmatrix} \rightsquigarrow \mathbf{w}^{(t'')} = \begin{bmatrix} 2.36 \\ 4.43 \\ 3.13 \\ -2.1 \\ 1.98 \\ -1.2 \end{bmatrix}$$

This means that  $\hat{R}(\mathbf{w}^{(t'')}) \leq \hat{R}(\mathbf{w}^{(t')}) \leq \hat{R}(\mathbf{w}^{(0)})$ , for instance

$$\hat{R}(\mathbf{w}^{(0)}) = 100 \rightsquigarrow \hat{R}(\mathbf{w}^{(t')}) = 0.01 \rightsquigarrow \hat{R}(\mathbf{w}^{(t'')}) = 0.001$$

## Training by Penalized Risk: *Regularization*

We may note that as *training progresses*, vector  $\mathbf{w}^{(t)}$  becomes *larger*

+ What if we add a *penalty proportional to  $\|\mathbf{w}^{(t)}\|^2$*  to risk?

$$\tilde{R}(\mathbf{w}) = \hat{R}(\mathbf{w}) + \|\mathbf{w}\|^2$$

Let's look at the *penalized risk* at *different iterations*

$$\tilde{R}(\mathbf{w}^{(0)}) = \hat{R}(\mathbf{w}^{(0)}) + \|\mathbf{w}^{(0)}\|^2 = 100$$

$$\tilde{R}(\mathbf{w}^{(t')}) = \hat{R}(\mathbf{w}^{(t')}) + \|\mathbf{w}^{(t')}\|^2 = 19.04$$

$$\tilde{R}(\mathbf{w}^{(t'')}) = \hat{R}(\mathbf{w}^{(t'')}) + \|\mathbf{w}^{(t'')}\|^2 = 44.761$$

If we minimize *penalized risk*

we *may get* from  $\mathbf{w}^{(0)}$  to  $\mathbf{w}^{(t')}$ , but will *never* get from  $\mathbf{w}^{(t')}$  to  $\mathbf{w}^{(t'')}$

## Training by Penalized Risk: *Regularization*

### Regularization

In *training* with *regularization*, we minimize a *penalized (regularized)* form of the empirical risk, i.e.,

$$\min_{\mathbf{w}} \hat{R}(\mathbf{w}) + \Pi(\mathbf{w}) \quad (\text{Regularized Training})$$

$\Pi(\mathbf{w})$  is a penalty that describes the behavior of  $\mathbf{w}$  in the case of *overfitting*

## Classical Regularization Approaches

Some **well-known** regularization approaches are

- $\ell_2$  or **Tikhonov regularization** in which we add a term proportional to  $\|\mathbf{w}\|^2$

$$\Pi(\mathbf{w}) = \lambda \|\mathbf{w}\|^2$$

- ↳ This way we **avoid** very **large weights**
- ↳ This **prevents perfect fit** to **training** dataset **reducing** chance of **overfitting**
- ↳ It is also called **weight decay**  $\rightsquigarrow$  Check why!

- $\ell_1$  or **Lasso regularization** in which we add a term proportional to  $\|\mathbf{w}\|_1$

$$\Pi(\mathbf{w}) = \lambda \|\mathbf{w}\|_1 = \lambda \sum_{i=1}^D |w_i|$$

- ↳ This way enforce  $\mathbf{w}$  to be **sparse**, i.e., to have to many zeros
- ↳ This way we **reduce** the **capacity of NN** and thus prevent **overfitting**

## Regularizing by *Dropout*

A less conventional **regularization** approach is **dropout** that was proposed by Hinton et al. first in *their 2012 paper* and then in *their 2014 paper*<sup>4</sup>

for each **training** iteration, we **deactivate** some **nodes** of NN **at random**  
or in other words we **drop them out**

### Intuition Behind Dropout

**Overfitting** means that NN is **larger** than **required**  $\rightsquigarrow$  With **dropout**

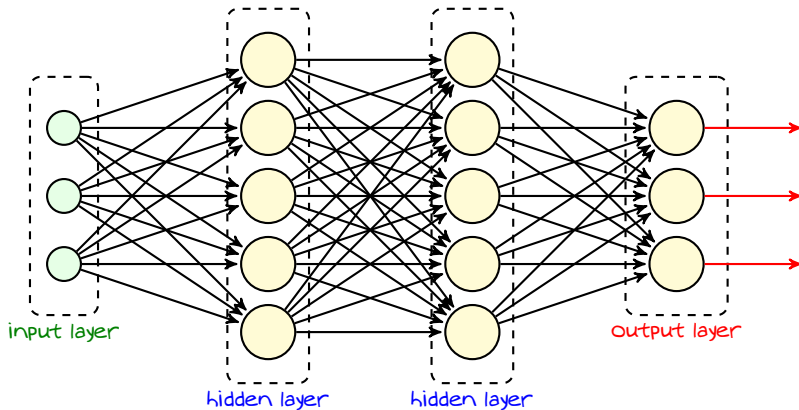
- we **randomly** switch among **smaller sub-models**
- many of these **smaller models** **do not overfit**
- the NN somehow represents average of these **not-overfitting models**

---

<sup>4</sup>Click to check out the papers!

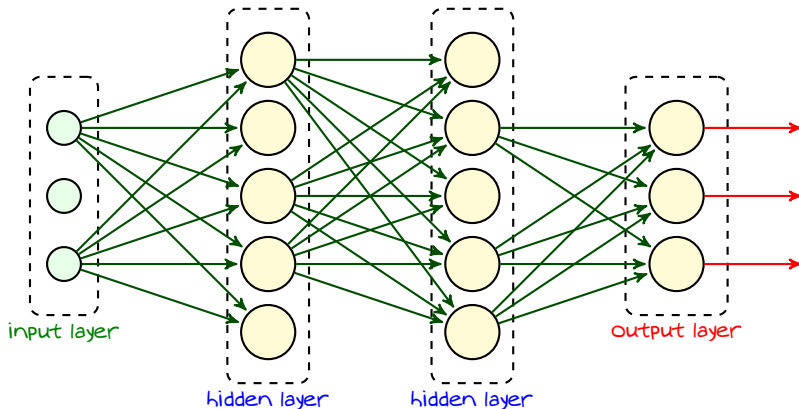
## Dropout: *Schematic*

Let's say this is our *NN*



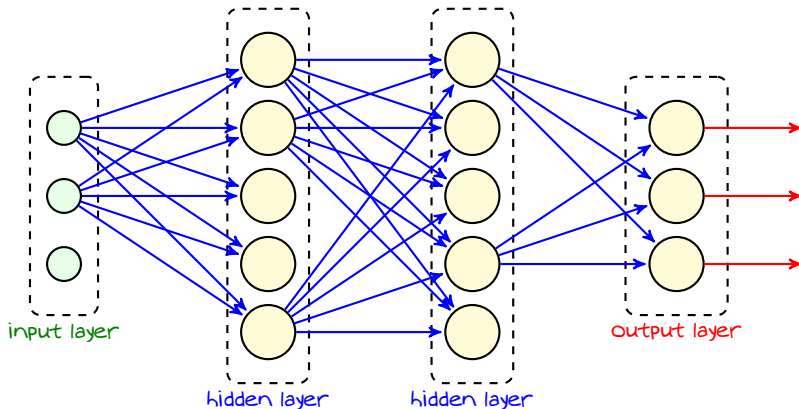
## Dropout: Schematic

For *first sample* we select few *nodes* in *each layer*



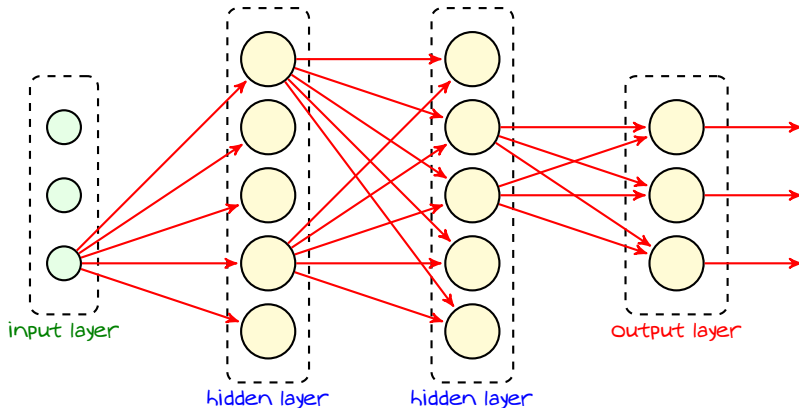
## Dropout: Schematic

For *second sample* we select new *nodes* in *each layer* at random



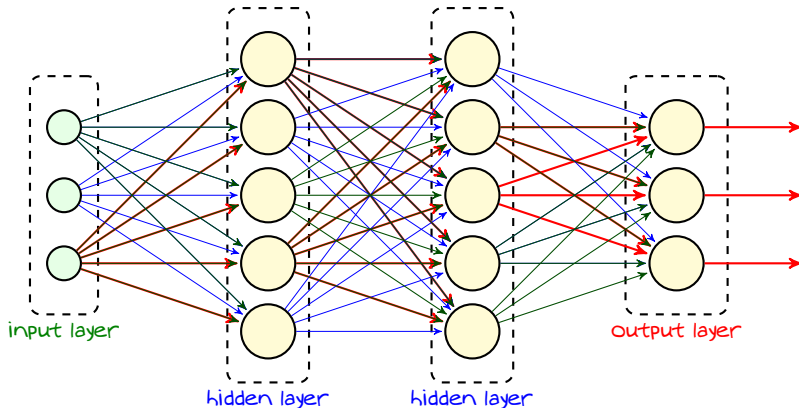
## Dropout: Schematic

For *next sample* we select again few *nodes* in *each layer* at random



## Dropout: *Schematic*

At the end, we *average* the gradients determined over these *smaller models*



## Dropout: *Training Loop*

Training with **dropout** is as before with a **minor change**

For *each sample* pass, we *randomly mask* a *portion of neurons* in each layer

When we pass **forward**, we generate **random masks** for each **layer**  $\ell = 0, \dots, L$

- Mask of **layer**  $\ell$  is a **same-size array** with **binary entries**, i.e.,  $\mathbf{s}_\ell \in \{0, 1\}^{\mathcal{W}_\ell}$
- Entries of  $\mathbf{s}_\ell$  are generated **randomly**

$$\text{each entry of } \mathbf{s}_\ell = \begin{cases} 1 & \text{with probability } p_\ell \equiv \text{retention probability} \\ 0 & \text{with probability } 1 - p_\ell \equiv \text{dropout probability} \end{cases}$$

### Layer Mask

With **random mask**  $\mathbf{s}_\ell = \text{mask}(\mathcal{W}_\ell | p_\ell)$ , we can mask layer output  $\mathbf{y}_\ell$  as

$$\mathbf{y}_\ell \leftarrow \mathbf{y}_\ell \odot \mathbf{s}_\ell$$

## Dropout: *Forward Propagation*

DropoutForwardProp():

```
1: Initiate with $\mathbf{y}_0 = \mathbf{x}$
2: for $\ell = 0, \dots, L$ do
3: Generate $\mathbf{s}_\ell = \text{mask}(\mathcal{W}_\ell | p_\ell)$ # random mask
4: Set $\mathbf{y}_\ell = \mathbf{y}_\ell \odot \mathbf{s}_\ell$ # mask output
5: Add $y_\ell[0] = 1$ and determine $\mathbf{z}_{\ell+1} = \mathbf{W}_{\ell+1} \mathbf{y}_\ell$ # forward affine
6: Determine $\mathbf{y}_{\ell+1} = f_{\ell+1}(\mathbf{z}_{\ell+1})$ # forward activation
7: end for
8: for $\ell = 1, \dots, L + 1$ do
9: Return $\mathbf{y}_\ell$ and $\mathbf{z}_\ell$
10: end for
```

Backpropagation goes *exactly as before*: of course masked *outputs* participate with value *zero* in gradient computation

## Training vs Evaluation

- In *training* we use random mask  $\rightsquigarrow$  `model.train()`
- In *evaluation* we *don't* use random mask  $\rightsquigarrow$  `model.eval()`

## Dropout: *Backpropagation*

Two points about dropout to do some research on!

### Layer Re-scaling

After training, we *re-scale* weights of each layer with *retention rate*

$$\mathbf{W}_\ell \leftarrow p_\ell \mathbf{W}_\ell$$

A similar approach is included in all classic implementations *but during training*

? Check how *PyTorch* does the re-scaling

### Equivalence to Weight Decay

If we apply *dropout at the input layer* of a *linear regression* model, it behaves in expectation like  *$\ell_2$ -norm regularization*



UNIVERSITY OF  
TORONTO

## Further Read

## Further Read

### Read More on These Topics



Goodfellow, et al., *Deep Learning*

↳ [Chapters 5 and 7](#)



Bishop & Bishop, *Deep Learning: Foundations and Concepts*

↳ [Chapter 7](#)

### Nice to Check

1. *A Direct Adaptive Method for Faster Backpropagation Learning: RPROP*  
↳ M. Riedmiller and H. Braun (1993)
2. *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*  
↳ N. Srivastava et al. (2014)